

ClaudeFast v5.3 Quick Start Guide

Read this first. You will be building in under 5 minutes.

What You Downloaded

```
your-project/
├─ .claude/
│   ├─ agents/           # 18 specialized AI experts (Sonnet + Opus tiers)
│   ├─ skills/           # 25+ domain knowledge bases
│   ├─ hooks/            # 6 intelligent automation hooks
│   ├─ rules/            # Repo-specific context (auto-loaded)
│   │   └─ repo-primer.md # Fill in on first run
│   ├─ commands/         # /team-plan, /build, /team-build, /view, /library
│   ├─ tasks/            # Living documentation & sessions
│   ├─ library.json       # Not included, created on /library setup
│   └─ settings.json      # Hook config, status line, permissions
├─ CLAUDE.md             # Mission control
├─ justfile              # Shorthand commands (optional)
└─ [your code]           # Production-ready from day one
```

Drop the `.claude` folder, `CLAUDE.md`, and `justfile` into the root of any project. That is the entire installation.

Setup (One Time)

1 Extract to your project

Copy the `.claude` folder, `CLAUDE.md`, and `justfile` to your project root. If you already have a `CLAUDE.md`, merge the contents or rename your existing one.

2 Install the Permission Hook

```
npm i -g @abdo-el-mobayad/claude-code-fast-permission-hook
```

This enables LLM-powered auto-approval so you are not interrupted by permission dialogs every few seconds.

3 Configure the Permission Hook

```
cf-approve install && cf-approve config
```

You will be prompted for your OpenRouter API key. The hook uses a small, fast model to classify operations as safe/dangerous/edge-case. Safe operations get instant approval. Dangerous ones get blocked. Edge cases get analyzed.

4 Activate hooks and status line

The included `.claude/settings.json` contains the full configuration for all hooks (Skill Activation, Context Recovery, Formatter, Library Sync) plus the status line monitor. You have two options:

- **Option A (Recommended):** If you do not have an existing `.claude/settings.json`, the included one works as-is. You are done.
- **Option B:** If you already have a `.claude/settings.json`, merge the `"hooks"` and `"statusLine"` sections from the included file into your existing one.

The `settings.json` activates four hooks plus the status line:

- **Skill Activation Hook** (UserPromptSubmit) — Auto-appends skill recommendations to every prompt. This is what guarantees 100% skill adherence.
- **Context Recovery Hook** (PreCompact) — Automatically backs up your session state before compaction events. Token-based triggers fire at 50k tokens used, then every 10k after.
- **Formatter Hook** (PostToolUse) — Runs Prettier on files after Write/Edit operations.
- **Library Sync Hook** (Stop hook) — Auto-syncs library-managed file edits at the end of every Claude turn. Single synchronous push, no detached worker. Inert until you set up a central library (see Step 9).
- **Status Line** — Shows real-time context usage, API budget, peak/off-peak indicator, and backup status directly in your Claude Code interface.

5 Install Just (optional shorthand)

The included `justfile` provides shorthand commands for launching Claude Code. For example, `just team` is a quick way to run the Agent Teams environment variable command. Install `just` if you want to use these shortcuts:

```
# macOS
brew install just

# Windows (via scoop)
scoop install just

# Or via cargo
cargo install just
```

This is optional. You can always pass the environment variable directly instead (see "Launching with Agent Teams" below).

6 Fill in the Repo Primer (first run)

ClaudeFast separates **behavioral instructions** from **repo-specific context**:

- **CLAUDE.md** — Purely behavioral. Defines how Claude should work: routing rules, agent coordination, coding standards, and the framework protocol. You should not need to edit this.
- **.claude/rules/** — Repo-specific context. Anthropic's rules directory feature automatically appends all files in this folder to CLAUDE.md within the AI's context window. Same effect as merging the files, but keeps things organized.

Inside `.claude/rules/` you will find `repo-primer.md` -- a structured template with sections for your tech stack, project structure, database, auth, build commands, environment variables, and common gotchas. On your first run in a new repo, ask Claude to fill it in:

```
"Explore this codebase and fill in the repo primer template at .claude/rules/repo-primer.md"
```

Claude will scan your project and populate the template with the right details. This gives every future session instant context about your repo without you needing to explain it each time.

7 Open Claude Code and start building

```
claude
```

That is it. Start prompting naturally. ClaudeFast handles the rest.

8 Install CodeStats (optional, for codebase analysis)

```
pip install cf-codestats
```

Requires Python 3.10+. The PyPI package is `cf-codestats` but the CLI command is `codestats`. It provides dependency graph analysis, hotspot scoring, dead code detection, and bus factor calculations. The `codestats` skill uses this CLI tool and is automatically recommended by the planning pipeline for large refactors and migrations. If you do not install it, everything else works normally -- this skill simply will not have data to work with.

9 Set up a Central Library (optional, for multi-project sync)

If you work across multiple projects, the Central Library system lets you manage your `.claude` folder from a single repository. Skills, agents, hooks, commands, and settings live in one place. Changes in one project sync to all others automatically.

Quick setup (recommended):

The included `/library` command teaches Claude the full library structure and workflows. Just run it with a natural language request:

```
/library let's set up my own library
```

Claude will create your private repo from the template, install the auto-sync hook, import your existing `.claude/` content, and connect everything.

Manual setup:

1. Fork the public template at github.com/Abdo-El-Mobayad/claude-fast-library
2. Clone your fork locally
3. From the library directory, run `node sync.mjs --link` once per device. This registers the local clone path in `~/.claude/library-paths.json` so the LibraryHook can find it. Repeat `--link` on every machine you work from.
4. Run `node sync.mjs --seed --name "CLAUDE--my-project"` to import your existing setup into the library
5. Run `node sync.mjs --init --profile dev` on new projects to pull in a predefined bundle of items

Profiles and Variants:

Profiles are reusable bundles of skills, agents, commands, and hooks defined in `map.json`. Initialize a new project with `--init --profile dev` to get your standard web dev stack, or create custom profiles for different project types.

Variants let the same item have project-specific versions using a `name--suffix` convention. For example, `react-strict` deploys as `react/` but with stricter rules. The suffix is never visible in the deployed project.

Everything works through natural language:

The `/library` command is a natural language interface. Claude understands the full library structure, all operations, and your profiles. Some examples:

```
"Hey Claude, this is a new project -- pull in my basic web dev profile claude setup from my library"

"Hey Claude, this new skill we built is excellent, add it to my library for the Dev profile"

"What's out of sync between this project and my library?"

"Push my changes back to the library"

"Create a variant of the react skill called react--strict for this project"

"Show me everything in my library"
```

This is completely optional. Without a manifest file, all Library hooks are inert. Your Code Kit works fully without it.

Your First Prompt

Once Claude Code is open, try something like:

```
"Build me a landing page with a hero section and email signup form"
```

Here is what happens behind the scenes:

1. **Skill Activation Hook fires** — Analyzes your prompt and appends relevant skill recommendations (e.g., frontend-design, session-management)
2. **Central AI reads CLAUDE.md** — Understands the framework architecture, routing rules, and your project context
3. **Complexity assessment** — Classifies your request as trivial, moderate, or complex
4. **Agent dispatch** — Routes work to the right specialist (Frontend Specialist for UI, Backend Engineer for APIs, etc.)
5. **Permission Hook handles approvals** — File reads, writes, and safe commands are auto-approved
6. **You get production-ready code** — Formatted, linted, and following established patterns

The Status Line

Once configured, you will see a live status line at the top of your Claude Code session:

```
* Actualizing... (2m 35s · ↓ 3.6k tokens)

>

Opus 4.6 | 70k / 200k | 35% used 69,855 | 49% free 97,144 | think: On
current: ●○○○○○ 10% | weekly: ●○○○○○ 25% | extra: ○○○○○○ $0.00/$30.00
resets 3:00am (1h36m) | resets Fri, 3:00pm | resets apr 1
-> .claude/backups/38-backup-10th-Mar-2026-1-12am.md
```

The ClaudeFast status line shows model info, context usage (tokens used / total), percentage free, API budget consumption (current session and weekly), budget reset times, and the latest backup file path.

The status line is powered by the Context Recovery Hook's status monitor. It updates in real time as you work, so you always know how much context you have left and when your next budget reset occurs.

Context Recovery and Backups

The Context Recovery Hook works entirely on its own. You do not need to configure or trigger it manually.

How it works: Before compaction events (when Claude's context window fills up and needs to be summarized), the hook automatically parses your conversation transcript, extracts the important decisions, progress, and context, and saves a structured backup to:

```
.claude/backups/1-backup-12th-Feb-2026-4-59pm.md
```

Backups are numbered sequentially (1, 2, 3...) and timestamped. Each backup contains your user requests, Claude's key responses, decisions made, and current progress. When you resume a session or start a new one, this context is available for Claude to pick up where you left off.

Tip: You can read your backup files at any time to review what happened in previous sessions. They are plain markdown files in `.claude/backups/`.

How Work Gets Routed

ClaudeFast uses a 4-tier complexity system. You do not need to think about this - it happens automatically based on CLAUDE.md instructions.

Complexity	Example	What Happens
Trivial	"Fix this typo"	Claude fixes it directly
Moderate	"Add a contact form"	Dispatches to Frontend Specialist
Complex	"Build user auth with dashboard"	Auto-invokes /team-plan, waits for your approval, then /build
Collaborative	"Build full-stack feature with API + UI + DB"	Auto-invokes /team-plan, waits for approval, then /team-build

Tip: For complex requests, Claude will show you a plan and wait for your approval before executing. You always stay in control.

Key Things to Know

Skills load automatically

You do not need to manually load skills. The Skill Activation Hook detects what your prompt needs and loads the right knowledge bases. Working with Supabase? The postgres-best-practices skill loads. Building a React component? The react performance rules load. It is all automatic.

Context is managed for you

Central AI conserves its context window by delegating work to specialist sub-agents. Each sub-agent gets a fresh context window, loads everything it needs, does the work, and returns results. You get 6x more effective building time before compaction.

CLAUDE.md is behavioral, rules/ is contextual

You should rarely need to edit `CLAUDE.md`. It contains the framework's behavioral instructions. Your project-specific context lives in `.claude/rules/repo-primer.md`, which the AI fills in on its first run. This separation means the framework can be updated independently of your repo context.

Specialists run on a model tier

Sub-agent files declare their default model in YAML frontmatter. v5.3 ships 13 specialists on Sonnet (focused, well-bounded work) and 5 on Opus (orchestration, creative, debugging). Override up to Opus per task when the work is critical, highly important, or highly challenging. You don't need to think about this for most prompts.

The framework evolves with you

When you discover a workaround or useful pattern, Claude will prompt you to capture it as a reusable skill. Your framework gets smarter over time. Use the `agentic-builder` skill to create custom skills, agents, and slash commands for your

specific project needs.

Sessions are documented automatically

Every decision, task, and architectural choice is tracked in session files under `.claude/tasks/`. Come back after a week and Claude can pick up where you left off.

Working with Complex Tasks

For simple work, just prompt naturally. Claude handles it directly or dispatches to the right specialist agent.

For more elaborate, multi-step work, use the `/team-plan` command followed by your prompt:

```
YOUR PROMPT IN CLAUDE CODE:
```

```
/team-plan Build a SaaS dashboard with user authentication, Stripe billing, and an admin panel for managing users
```

This generates a structured plan file in `.claude/tasks/` with task breakdowns, specialist assignments, dependencies, and acceptance criteria. Claude will show you the plan and wait for your approval.

Once you approve, follow up with one of two execution commands:

```
OPTION A - ISOLATED SUB-AGENTS (LOWER COST):
```

```
/build .claude/tasks/saas-dashboard-plan.md
```

```
OPTION B - COLLABORATIVE AGENT TEAMS (HIGHER COST, BETTER COORDINATION):
```

```
/team-build .claude/tasks/saas-dashboard-plan.md
```

When to use which

- **/build** — Each specialist agent works independently on their assigned task. Best for work where tasks do not depend on each other. Uses standard token cost.
- **/team-build** — Agents coordinate with each other in real time using Agent Teams. Best for complex cross-domain work where the frontend needs to know the API schema the backend is building, or the database design affects multiple services. Uses 2-4x more tokens, but produces better results for work requiring significant coordination.

Important: Agent Teams are not necessarily better for all cases. For focused, independent tasks, `/build` produces great results at lower cost. Reserve `/team-build` for genuinely cross-domain work where agents need to agree on shared interfaces.

Launching with Agent Teams

`/team-build` requires Agent Teams to be enabled for your session. Start Claude with the environment variable set:

```
POWERSHELL:
```

```
$env:CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS = "1"; claude
```

```
BASH / MACOS / LINUX:
```

```
CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1 claude
```

The included `justfile` provides a shorthand for this. If you have `just` installed (see Step 5), you can simply run:

```
SHORTHAND (USING JUSTFILE):  
  
just team
```

All methods do the same thing: set `CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1` for that session only. They do not modify your global settings. Once your session is started with Agent Teams enabled, you can use `/team-build` within it.

Tip: The justfile also provides `just cc` as a shorthand for launching Claude, and `just commands` to list available slash commands.

Other Useful Commands

Command	When to Use
just cc	Launch Claude Code (standard)
just team	Launch Claude with Agent Teams enabled
just commands	List all available slash commands
/view	Generate HTML visualization of a plan
/library	Manage your central library
/commit	Create a well-formatted git commit
Ctrl+T	Toggle task list visibility during work

Need Help?

- **Full product guide:** See the Code Kit README PDF for comprehensive documentation of all agents, skills, hooks, and capabilities.
- **What changed:** See the What's New in v5.3 PDF for a summary of everything added in this release.
- **Website:** claudefa.st for blog posts, guides, and the latest updates.